

OpenAI API 入門

從 API Key 到第一個 Python AI 函式

第03週 | 生成式AI應用開發

本週要完成什麼

第 2 週練過的 Python 技能，這週接上真實的 LLM API。

- 安全讀取 `OPENAI_API_KEY`
- 建立 OpenAI client
- 用 **Responses API** 送出第一個請求
- 讀取文字輸出、`response.id`、`usage`
- 封裝成可重複使用的 Python 函式
- 加入基本錯誤處理

本週成果：完成第一個 Python AI 問答程式

本週學習目標

#	能力	對應後續課程
1	安裝與匯入 OpenAI SDK	所有 API 實作基礎
2	用 Colab Secrets 讀取 API key	安全管理金鑰
3	呼叫 <code>client.responses.create()</code>	文字生成應用
4	使用 <code>instructions</code> 與 <code>input</code>	system prompt / user prompt
5	解析 <code>output_text</code> 與 <code>usage</code>	顯示回答、估算成本、除錯
6	封裝 API 呼叫函式	後續 App 開發
7	加入錯誤處理	網路、金鑰、額度與格式問題

開發流程：一次 API 呼叫的路徑

設定 API Key (Colab Secrets / .env)

↓

建立 `client = OpenAI()`

↓

`client.responses.create(model, instructions, input)`

↓

<code>response.output_text</code>	← 回答文字
<code>response.id</code>	← 這次呼叫的識別碼
<code>response.usage</code>	← token 使用量 (估成本)

↓

封裝成函式 → 加上 `try/except` 錯誤處理

第一節：安裝與匯入 SDK

Colab 每次重開 runtime 都要重新安裝套件。

```
# 安裝 OpenAI Python SDK
!pip install openai --quiet

# 匯入本週使用的模組
import os
import json
from openai import OpenAI
```

第二節：API Key 安全守則

- 不要把 API key 寫死在程式碼裡
- 不要把含 key 的檔案上傳到 GitHub
- Colab 用 **Secrets**；本機用 `.env` + `python-dotenv`

❌ 錯誤：金鑰寫死

```
client = OpenAI(api_key="sk-abc123...")
```

✅ 正確：從環境變數讀取

```
client = OpenAI() # 自動讀 OPENAI_API_KEY
```

在 Colab 設定 API Key

左側 Secrets () 面板新增：

- Name : OPENAI_API_KEY
- Value : 你的金鑰，並打開 Notebook access

```
# 從 Colab Secrets 讀取，轉成環境變數
from google.colab import userdata
os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")
```

本機環境可改用 `.env`：

```
OPENAI_API_KEY=sk-... + load_dotenv()
```

第三節：建立 Client 與選模型

`OpenAI()` 會自動從環境變數 `OPENAI_API_KEY` 讀取金鑰。

```
client = OpenAI()

# 課堂預設用較低成本的 mini 模型
# 可用環境變數 OPENAI_MODEL 覆蓋
MODEL = os.getenv("OPENAI_MODEL", "gpt-5.4-mini")

print(f"Client 建立完成，使用模型：{MODEL}")
```

費用控管：課堂範例一律以 **mini 等級** 模型為主

第四節：第一個 Responses API 呼叫

```
response = client.responses.create(  
    model=MODEL,  
    input="請用繁體中文，用三句話解釋什麼是生成式 AI 。"  
)  
  
print(response.output_text)
```

- `input`：要問的內容
- `response.output_text`：模型的回答文字

檢查 response 物件

不只看回答，也要知道呼叫細節。

```
print("response id:", response.id)
print("model:", response.model)
print("usage:", response.usage)
```

欄位	用途
output_text	顯示給使用者的回答
id	這次呼叫的識別碼（多輪對話會用到）
usage	token 使用量，用來估算成本

第五節：instructions（角色 / 規則）

`instructions` 就是 **system prompt**：放角色設定與規則。

```
response = client.responses.create(  
    model=MODEL,  
    instructions="你是一位資管系 Python 助教。回答精簡具體，用繁體中文。",  
    input="請解釋 API key 為什麼不能直接寫在程式碼裡。"  
)  
  
print(response.output_text)
```

同一個問題，有沒有 `instructions`，回答風格差很多

第六節：role / content 格式

除了 `instructions + input`，也能用訊息清單。

```
messages = [  
    {"role": "developer", "content": "你是教學助理，回答適合初學 Python 的大學生。"},  
    {"role": "user", "content": "請用生活化例子說明什麼是 API。"}  
]  
  
response = client.responses.create(model=MODEL, input=messages)  
print(response.output_text)
```

- `developer`：開發者設定的規則（等同 system）
- `user`：使用者的輸入

instructions vs messages：何時用哪個？

情境	建議寫法
輸入一段文字、得到一次回答	<code>instructions</code> + <code>input</code> 字串
需要多個角色訊息、對話結構	<code>input</code> 用 <code>role/content</code> 清單
需要 few-shot 給多個範例	<code>role/content</code> 清單

兩種都會用到，先熟悉最簡單的 `instructions + input`

第六節之二：多輪對話

Responses API 用 `previous_response_id` 接續上一輪。

```
response1 = client.responses.create(  
    model=MODEL, input="我正在學習 Python API 開發。"  
)  
  
response2 = client.responses.create(  
    model=MODEL,  
    previous_response_id=response1.id, # 接上一輪  
    input="根據上一句，請建議我下一步該練什麼。"  
)  
print(response2.output_text)
```

不必自己保存整段歷史，交給 `previous_response_id`

第七節：封裝成函式

每次手寫 `create()` 會讓程式很亂，包成函式。

```
def ask_ai(user_input, role="你是一位有幫助的助理", model=MODEL):  
    response = client.responses.create(  
        model=model,  
        instructions=role,  
        input=user_input  
    )  
    return response.output_text  
  
print(ask_ai("請用 3 個重點說明 Python 函式的用途。"))
```

這個 `ask_ai()` 之後的 App 開發會一直用到

第八節：基本錯誤處理

API 可能因金鑰、網路、額度而失敗，用 `try/except` 包起來。

```
def ask_ai_safe(user_input, role="你是一位有幫助的助理", model=MODEL):  
    try:  
        response = client.responses.create(  
            model=model, instructions=role, input=user_input  
        )  
        return True, response.output_text  
    except Exception as e:  
        return False, f"API 呼叫失敗：{e}"
```

回傳 (是否成功, 內容)，App 才不會整個當掉

第九節：讀取 usage、估算成本

```
def ask_ai_with_metadata(user_input, role="你是一位有幫助的助理", model=MODEL):  
    response = client.responses.create(  
        model=model, instructions=role, input=user_input  
    )  
    return {  
        "id": response.id,  
        "model": response.model,  
        "answer": response.output_text,  
        "usage": response.usage,    # input / output token 數  
    }
```

`usage` × 官方單價 = 估算費用；價格會變動，以當天 pricing 為準

第十節：課堂練習

練習	內容
A	Python 作業批改助教：改 <code>instructions</code> ，指出錯誤原因、修正方式、修正版程式
B	<code>usage</code> 與費用估算：把回應的 token 數接到 <code>calculate_cost()</code>
C (選做)	自由設計角色與 <code>instructions</code> ，比較不同角色的回答差異

目標：把 API 呼叫、函式封裝、`usage`、`instructions` 全部串起來

補充：三平台對照總表

同樣的「呼叫 LLM」，三家寫法略有不同。

項目	OpenAI	Gemini	Claude
套件	<code>openai</code>	<code>google-genai</code>	<code>anthropic</code>
金鑰	<code>OPENAI_API_KEY</code>	<code>GEMINI_API_KEY</code>	<code>ANTHROPIC_API_KEY</code>
建立 client	<code>OpenAI()</code>	<code>genai.Client()</code>	<code>anthropic.Anthropic()</code>
呼叫	<code>responses.create()</code>	<code>interactions.create()</code>	<code>messages.create()</code>
角色設定	<code>instructions</code>	<code>system_instruction</code>	頂層 <code>system</code>
輸入	<code>input</code> 字串 / 清單	<code>input</code> 字串	<code>messages</code> 清單
取回答	<code>output_text</code>	<code>output_text</code>	<code>content[0].text</code>
預設模型	<code>gpt-5.4-mini</code>	<code>gemini-3.5-flash</code>	<code>claude-haiku-4-5</code>

補充：Gemini API 寫法

```
from google import genai

client = genai.Client()          # 讀 GEMINI_API_KEY
MODEL = "gemini-3.5-flash"

interaction = client.interactions.create(
    model=MODEL,
    system_instruction="你是 Python 助教，用繁體中文精簡回答。",
    input="請解釋什麼是 API。"
)
print(interaction.output_text)
```

與 OpenAI 幾乎對應：`interactions.create()`、`system_instruction`、`output_text`

補充：Claude API 寫法

```
import anthropic

client = anthropic.Anthropic() # 讀 ANTHROPIC_API_KEY
MODEL = "claude-haiku-4-5"

message = client.messages.create(
    model=MODEL,
    max_tokens=512,
    system="你是 Python 助教，用繁體中文精簡回答。",
    messages=[{"role": "user", "content": "請解釋什麼是 API。"}],
)
print(message.content[0].text)
```

差異最大：一定要給 `max_tokens`、輸入用 `messages` 清單、取值用 `content[0].text`

補充：三平台的多輪對話

平台	多輪對話做法
OpenAI	傳 <code>previous_response_id</code> 接續上一輪
Gemini	傳 <code>previous_interaction_id</code> 接續上一輪
Claude	每次送出完整 <code>messages</code> 歷史 (自己保存對話)

OpenAI / Gemini：由平台記住上一輪，只送新訊息
Claude：由你的程式保存整段歷史，每次一起送出

補充：三平台特色與選用建議

平台	特色	適合情境
OpenAI	教材 / 社群最完整、生態成熟	課程主線、通用開發
Gemini	Google 生態、多模態、有免費額度	無信用卡先試、圖片影音
Claude	長文件理解、指令遵循穩定	長文摘要、嚴謹格式輸出

課程一律以 **OpenAI** 為主線；Gemini / Claude 僅作補充比較
有興趣可於自主學習週（Week 17–18）延伸
學好一個平台，換平台只需半天

本週小結

技能	學了什麼	之後用在哪
API key	Colab Secrets / 環境變數	所有 API 呼叫
Client	<code>client = OpenAI()</code>	建立連線
Responses API	<code>responses.create()</code>	文字生成主線
輸入格式	<code>input</code> 字串 / role-content	文字生成應用
輸出解析	<code>output_text</code> 、 <code>id</code> 、 <code>usage</code>	顯示、除錯、追蹤
函式封裝	<code>ask_ai()</code> 、 <code>ask_ai_safe()</code>	Streamlit App
錯誤處理	<code>try / except</code>	API 穩定性

課後動手做

完成本週三題課堂練習：

A Python 作業批改助教

B `usage` 與費用估算

C (選做) 自由設計角色與 `instructions`

把 API 呼叫、函式封裝、`usage`、`instructions` 全部串起來